PRIME-FLEET · SPÉCIFICATIONS V2

Prime Fleet

Application multiplateforme (PC, Android, iOS) qui pilote une flotte de PC Prime Agent Studio par Tailscale. Une conversation appartient à un projet, pas à une machine. La Roadmap se travaille avec toute la flotte, et elle accueille les campagnes de test multijoueur. L'app est livrée avec un jeu de skills, aussi utilisable depuis Studio de bureau.

Dépôt	`fabsther/prime-fleet`, public, licence MIT
Technologie	React + Vite + Tauri 2 (Windows, macOS, Linux, Android, iOS)
Base serveur	Studio 4.3.1+, API publique `/api/v1` uniquement
Flotte de test	ALIEN18, BAGUETTE · tablette Xiaomi Pad 6 (Android 14) · iOS plus tard
Recette finale	Campagne multijoueur Pieton (hôte ALIEN18, invité BAGUETTE)
Date	13 octobre 2026 · remplace les spécifications V1 (app Kotlin)

1. Objectifs

1. **Piloter à distance tous les PC de la flotte** depuis un PC, une tablette, un téléphone, et plus tard un appareil iOS.
2. **Les conversations appartiennent au projet**, pas à une machine. On peut les poursuivre sur n'importe quelle machine qui possède ce projet. L'interface indique quel agent, sur quelle machine, a écrit en dernier.
3. **Travailler la Roadmap avec la flotte** : assigner chaque tâche à une machine, puis la lancer sur cette machine.
4. **Campagnes de test multijoueur** : depuis la Roadmap, simuler une session avec au moins un joueur simili humain par machine, et jusqu'à deux par machine. L'app fournit l'infrastructure. Le jeu, le réseau et les actions sont décrits par chaque projet.
5. **Interface aussi proche que possible de Prime Agent Studio.**
6. **Jeu de skills Fleet** livré avec l'app, et utilisable dans Studio de bureau sans le modifier.

2. Contraintes non négociables

- **Toute interaction avec Studio passe exclusivement par l'API publique v1.** Aucune route interne, et aucune modification de Studio. Un besoin manquant devient une demande ou une PR au créateur, la plus petite possible.
- **Minimum de code** (méthode ponytail) : la plateforme avant les dépendances, aucune couche sans raison. Une revue ponytail a lieu à la fin.
- Les jetons ne sont jamais écrits en clair, ni commités, ni affichés dans les journaux.

3. Architecture

ÉLÉMENT	CHOIX
Interface	React + Vite. Style imposé : skill `prime-agent-studio-style` (mesuré sur Studio 4.3.2, MIT). On copie `studio.css` (jetons sombre et clair, classes `ps-`), `icons.esm.js` et `theme.js` avec leur licence. Les composants React reprennent le balisage de `example-shell.html` et `components.html`. Aucune autre bibliothèque d'interface.
Conteneur	Tauri 2, une seule base de code. Builds : Windows et Android en premier, iOS ensuite.
Couche réseau	Commandes Rust génériques : `api(machine, method, path, body)`, `stream(machine, path, after)` pour le SSE, et `download(machine, path, dest)`. **Pourquoi :** une webview ajoute toujours un en-tête `Origin`, et Studio refuse toute requête qui en porte un (`403 api_forbidden`, vérifié). La couche Rust appelle les mêmes routes, sans `Origin`.
Jetons	Coffre du système, avec un plugin keyring : Keystore sur Android, Keychain sur iOS et macOS, Credential Manager sur Windows. Les autres réglages (machines sans jeton, préférences d'affichage) sont dans un fichier JSON de l'app.
Ligne de commande	L'exécutable de bureau sert aussi d'outil en ligne de commande : `prime-fleet api <machine> <METHOD> <path> [json]` et `prime-fleet machines`. Il lit le coffre, appelle l'API, et écrit le JSON sur la sortie standard. Les skills s'en servent (section 10).
Topologie	Le client parle directement à chaque Studio, par Tailscale HTTPS (`https://<pc>.<tailnet>.ts.net`). Il n'y a ni serveur central, ni PC propriétaire. L'état partagé passe par la synchro de Studio.

4. API Studio utilisée (v1)

DOMAINE	ROUTES	USAGE
Machine	`GET /machine`	Nom, `machineId`, version, capacités (`computer-use`, puis `sync`). Les fonctions s'affichent selon ces capacités.
Projets	`GET /projects`	Regroupement par `syncId`.
Conversations	`GET /projects/{id}/sessions` · `GET /sessions/{id}/messages`	Union des conversations de toutes les machines, dédoublonnée par `id`.
Runs	`POST /projects/{id}/runs` · `GET /runs[/{id}]` · `/events` · `/stop` · `/interactions`	Continuer une conversation sur une machine, suivi en direct avec reprise, arrêt, questions de l'agent.
Message en direct	`POST /sessions/{id}/messages`	`steer` ou `follow_up`, vers la machine qui exécute le run.
Roadmap	`GET …/roadmap` · `POST …/roadmap/mutations` · `POST …/roadmap/work`	Lecture, édition avec `expectedRevision`, lancement d'une étape. Depuis 4.3.1, une étape cochée garde la machine, la conversation et l'heure.
Fichiers	`GET /sessions/{id}/files` · `GET/HEAD …/files/{fileId}`	Rapports des joueurs et livrables.
Computer Use	`GET /computer-use` · `POST /computer-use`	Lire l'état ; autoriser ou retirer le contrôle pour une session ou un run. Un seul propriétaire par machine. Scope `computer:write`.
Synchro (proposée par le créateur)	`POST /projects/{id}/sync {sessionId?}` · `GET /sync`	Avec `sessionId` : synchro ciblée de la conversation, réponse `{state, changed}` à la fin. Sans `sessionId` : cycle complet, réservé aux jetons qui ont accès à tous les projets synchronisés. Scope `sync:write`, décoché par défaut. `GET /sync` : configurée, en cours, dernier résultat.

Scopes des jetons de l'app : `read`, `runs:write`, `roadmaps:write`, `files:download`, `computer:write`, `sync:write`.

> Règles générales : chaque écriture porte un `requestId` ou un `expectedRevision`. Un 409 oblige à relire, jamais à rejouer en aveugle. L'idempotence dure 1 heure et n'est pas gardée si Studio redémarre : l'app ne relance jamais toute seule une action dont l'issue est ambiguë.

> Demandes au créateur sur la synchro : capacité `sync` dans `GET /machine` ; liste fixe des valeurs de `state` ; distinguer « conversation pas encore présente » et « autre projet » ; durée maximale d'attente, ou 409 si un cycle est déjà en cours ; confirmer le correctif « l'envoi après un run attend la fin du cycle en cours ».

5. Modèle de données (côté client)

OBJET	DÉFINITION
Machine	`machineId`, nom, adresse, capacités, état (en ligne / hors ligne / erreur), jeton (dans le coffre).
Projet de la flotte	Un groupe de projets Studio ayant le même `syncId` (sinon le même nom). Il garde, pour chaque machine, son `projectId`.
Conversation de la flotte	Un `sessionId` unique dans le projet. Elle connaît les machines qui en ont une copie, la copie la plus récente (`updatedAt`), le **dernier agent** (machine du dernier run ou de la copie la plus récente), et le run actif s'il y en a un.

6. Conversations indépendantes des machines

- La liste des conversations est rangée par projet. Chaque conversation affiche « dernier agent : BAGUETTE » et un indicateur s'il y a un run actif.
- L'historique est lu sur la copie la plus récente.
- **Sélecteur de machine** dans la zone de saisie. Il liste les machines en ligne qui ont ce projet. La machine du dernier agent est choisie par défaut.
- **Continuer sur une autre machine B** :
  1. Si un run est actif sur A, l'app propose « Attendre la fin » ou « Arrêter puis continuer sur B ».
  2. `POST …/sync {sessionId}` sur A, puis sur B. Le message « Synchronisation… » s'affiche pendant ce temps.
  3. `POST /projects/{pB}/runs {sessionId, message}` sur B.
- **Repli si la capacité `sync` manque** : l'app compare le dernier message sur A et sur B. S'ils diffèrent : « pas encore synchronisée sur B », et le bouton d'envoi est grisé.
- Pendant un run, le message part toujours vers **la machine qui exécute** (`steer` ou `follow_up`). Le sélecteur est verrouillé tant que le run tourne.
- Le reste est conservé de la V1 : suivi en direct avec reprise, questions de l'agent, arrêt, fichiers, nouvelle conversation sur une machine choisie.

7. Roadmap et markups

Ces conventions sont du texte écrit dans la Roadmap, que la synchro copie partout. Chaque directive occupe une ligne seule, sans tenir compte des majuscules.

7.1 `@machine:` — assigner une étape

```
Implémenter le menu coop
@machine: BAGUETTE
```

- Elle se place dans la note de l'étape. La valeur est le nom Studio ou le `machineId`. Une seule directive par note : assigner une autre machine la remplace, et l'app la réécrit en fin de note.
- « Lancer » : l'app relit la Roadmap sur la machine assignée, vérifie que l'étape y est (sinon elle lance une synchro du projet si c'est permis, ou affiche « pas encore synchronisé »), puis envoie `roadmap/work {plan, stepId}`.
- Une étape cochée affiche la machine qui l'a faite (donnée de Studio 4.3.1).

7.2 `@campaign` — campagne de test

```
@campaign
Commit attendu : 854ecb0… Hôte : ALIEN18 (100.118.205.36), UDP 7777.
L'invité réessaie toutes les 15 s pendant 10 min. Rapport : outputs/multitest/<id>/player-<k>.md
```

- Un plan dont le résumé contient `@campaign` est une campagne. Le reste du résumé forme les consignes communes.
- Chaque étape non cochée est un joueur. Le texte donne son rôle. La note commence par `@machine:`, suivie des actions numérotées, chacune avec un critère de réussite observable.
- Une ligne optionnelle `@gui` dans la note indique un joueur qui pilote le bureau (Computer Use). Il peut y en avoir au plus un par machine.

7.3 En-tête envoyé à chaque joueur

```
Campagne de test <campaignId> — joueur <k>/<N> : <rôle> (machine <M>).
Joueurs : 1) ALIEN18 Joueur 1 — hôte ; 2) BAGUETTE Joueur 2 — invité
Consignes communes :
<résumé du plan sans la ligne @campaign>
```

- Il est envoyé dans `instructions` de `roadmap/work` (4000 caractères au plus). Studio y ajoute le texte et la note de l'étape.
- `campaignId` : 12 caractères hexadécimaux. `requestId` : `cmp_<campaignId>_<stepId>`, coupé à 100 caractères.

7.4 Résultats

- Chaque joueur écrit `outputs/multitest/<campaignId>/player-<k>.md` et cite ce chemin dans sa réponse finale. L'app le récupère avec les fichiers de la conversation.
- La synthèse va dans le journal du plan. Les étapes ne sont cochées que si l'utilisateur valide.

8. Lancement d’une campagne et Computer Use

1. **Vérifications** : une machine connue par joueur, au plus 8 joueurs et 1 joueur `@gui` par machine, des consignes de 4000 caractères au plus.
2. **Pré-contrôle** : `GET /computer-use` sur chaque machine qui a un joueur `@gui`. Si un autre propriétaire détient le contrôle, l’app l’affiche et propose d’annuler. Elle ne retire jamais le contrôle d’une autre cible. Sur une machine sans la capacité, elle affiche un rappel manuel.
3. **Synchro du projet** sur chaque machine si c’est permis, pour que l’étape de chaque joueur existe partout.
4. **Envoi des joueurs** : l’un après l’autre sur un même PC, en relisant la révision avant chaque envoi ; en parallèle entre les PC.
5. **Autorisation automatique** : `POST /computer-use {enabled:true, expectedRevision, runId}` dès qu’un joueur `@gui` est accepté. Le joueur attend au plus 2 minutes que Computer Use soit disponible.
6. **Panneau de campagne** : pour chaque joueur, sa machine, l’état du run, l’état de Computer Use et l’erreur éventuelle, avec ses actions. « Message à tous ». « Tout arrêter » arrête les runs, puis retire Computer Use (`enabled:false`).
7. Aucun réessai automatique. L’erreur d’un joueur ne bloque pas les autres.

9. Interface et mise en page

- **Style** : skill `prime-agent-studio-style`. Jetons uniquement, sombre par défaut, accent lavande rare, texte 12 à 13 px, lignes de 38 px, statut = point de 6 px + un mot, icônes au trait, textes selon `references/writing.md` (vocabulaire Studio : Machine, Projet, Conversation, Run, Roadmap). Revue finale avec `references/checklist.md` et comparaison aux captures.
- **Barre latérale = Projets → Conversations** (`.ps-tree`, `.ps-row`, `.ps-subrow`). Chaque conversation porte un point de la couleur de la machine du dernier agent. *Écart voulu* au modèle « Machines → Projets » du skill : ici une conversation appartient au projet.
- **Fil d'Ariane** : Projet / Conversation, avec la machine du dernier agent en pastille. Le sélecteur de machine est dans le composer.
- **Écran Machines** : cartes `.ps-card` sur PC, liste `.ps-machine-list` sur mobile (état, adresse, capacités, runs actifs, Computer Use).
- **Zones libres avec onglets** (barres d'onglets de 35 px, séparateurs de 1 px) : chaque zone affiche n'importe quelle vue. Par défaut : conversation d'un côté, Roadmap du projet de l'autre.

APPAREIL	MISE EN PAGE
PC (Windows, macOS, Linux)	Comme Studio : barre latérale et zones de travail.
Tablette Android ou iOS	Deux zones au plus. **Paysage : partage vertical** (côte à côte). **Portrait : partage horizontal** (l'une au-dessus de l'autre). La liste des projets est dans un tiroir latéral.
Téléphone	Une seule zone, avec la navigation par défaut, **tant qu'il n'est pas branché sur un écran externe**. Sur un écran externe, la mise en page suit la taille de cet écran (tablette ou PC).

Règle simple : seuils du skill (≤ 760 px : mobile, barre latérale en tiroir, en-tête sur deux lignes ; ≤ 1250 px et ≥ 1600 px : largeurs ajustées), plus l'orientation pour le sens du partage sur tablette. Media queries et container queries, sans détection d'appareil. Zones de sécurité (`env(safe-area-inset-*)`) respectées.

10. Jeu de skills Fleet

Ce sont des skills Prime Agent standard, dans `prime-fleet/skills/`. Ils sont **chargés sans modifier Studio** : on ajoute ce dossier dans la liste `skills` des réglages de Prime Agent, sur chaque PC. Un `git pull` les met à jour.

SKILL	RÔLE
`fleet-machines`	Lister les machines, leur état et leurs capacités (`prime-fleet machines`, `GET /machine`).
`fleet-roadmap`	Assigner une étape avec `@machine:`, puis la lancer sur sa machine.
`fleet-conversation`	Continuer une conversation sur une autre machine (synchro, puis run).
`multiplayer-testing`	Format `@campaign` et `@gui`, liste à remplir par l'auteur (même commit, build, réseau, actions observables, rapport, fermeture, Computer Use), lancement et suivi. S'appuie sur les trois autres.

- Les scripts appellent uniquement la commande de l'app de bureau : `prime-fleet api <machine> <METHOD> <path> [json]`. Il n'y a donc aucun jeton en clair ni aucune dépendance Python. Prérequis : l'app de bureau est installée sur le PC.

- Les scripts Python n'utilisent que la bibliothèque standard. Toute action réelle (lancer, autoriser Computer Use) se fait seulement avec `--confirmed`. Sans cette option, le script fait une simulation.

11. Structure du dépôt

```
prime-fleet/
  app/            React + Vite (src/, studio.css + icons + theme.js du skill, LICENSE MIT)
  app/src-tauri/  Rust : commandes api/stream/download, keyring, CLI
  skills/         fleet-machines, fleet-roadmap, fleet-conversation, multiplayer-testing
  docs/           SPEC.md, DECISIONS.md
  LICENSE (MIT), README.md
```

12. Limites connues

LIMITE	TRAITEMENT
Un seul propriétaire de Computer Use par machine	Au plus un joueur `@gui` par PC. Les autres joueurs de ce PC jouent sans interface graphique.
Synchro pas encore livrée	L'app vérifie la capacité `sync`. Sans elle, elle compare le dernier message.
Idempotence non persistante (1 heure)	Aucun réessai automatique. L'utilisateur décide.
Machine éteinte	Elle s'affiche hors ligne, et rien n'est mis en file. Les données restent visibles via les copies synchronisées des autres machines.
Backend Computer Use et modèle	Ils se règlent dans Studio, sur le PC. Un modèle sans vision provoque un 409, qui est expliqué à l'utilisateur.
iOS	Le build est prévu, mais il n'est pas testé pour l'instant : il demande un Mac et Xcode.

13. Hors périmètre

Ajouter au projet une machine qui ne l'a pas (bouton « + », installation guidée par un `WorkspaceSetup.md` maintenu dans le dépôt et cité dans `AGENTS.md`) : reporté. L'API v1 ne permet pas d'enregistrer un projet sur une machine. Notifications push, mode démo, cache hors ligne avancé, choix du backend Computer Use depuis l'app, plusieurs joueurs graphiques sur un même PC, routes serveur propres à Fleet, ordres signés.

14. Livraison et critères d'acceptation

Tout est livré en une fois, mais construit par étapes internes, chacune testée.

1. **Socle** : Tauri, couche Rust, coffre, commande en ligne. Test : `prime-fleet api ALIEN18 GET /machine` et la même chose sur BAGUETTE.
2. **Machines et projets** regroupés. Test sur Windows et sur la tablette.
3. **Conversations de la flotte** : copies dédoublonnées, dernier agent, suivi en direct, questions, arrêt, fichiers, sélecteur de machine et poursuite sur l'autre PC (avec synchro si elle est disponible).
4. **Roadmap** : édition, `@machine:`, lancement, machine qui a fait une étape.
5. **Campagnes** et Computer Use.
6. **Skills**, chargés depuis les réglages de Prime Agent sur ALIEN18 : simulation, puis lancement.
7. **Mise en page** : PC, tablette en paysage et en portrait, téléphone avec et sans écran externe.

8. **Recette** : la campagne Pieton (hôte ALIEN18, invité BAGUETTE) produit les rapports `player-1.md` et `player-2.md`.
9. **Revue ponytail** finale. Le résultat est donné en lignes supprimables.

15. Leçons de l'environnement

- Android : utiliser le SDK déjà installé (`android-36.1`), et le JBR d'Android Studio pour `JAVA_HOME`.
- Xiaomi HyperOS : activer « Installer via USB » dans les options pour les développeurs.
- BAGUETTE est en 4.3.0, sans `computer-use` : le mettre à jour en 4.3.1 ou plus.
- Pieton : la première ouverture sur un PC compile le C++ puis les shaders (30 à 90 minutes). Prévoir des délais d'attente longs.